

```
import tensorflow as tf
print("GPU Available:", tf.config.list_physical_devices('GPU'))
```

```
GPU Available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# STEP 2: Define dataset paths (CORRECTED)
# This path is taken from the previously working code
# Note: There is a space before 'Project' in the folder name
```

```
import os
```

```
BASE_DIR = "/content/drive/MyDrive/ Project/archive (2)/dataset"
```

```
TRAIN_DIR = os.path.join(BASE_DIR, "TRAIN")
VAL_DIR    = os.path.join(BASE_DIR, "VAL")
TEST_DIR   = os.path.join(BASE_DIR, "TEST")
```

```
print("TRAIN directory exists:", os.path.exists(TRAIN_DIR))
print("VAL directory exists:", os.path.exists(VAL_DIR))
print("TEST directory exists:", os.path.exists(TEST_DIR))
```

```
print("\nTRAIN classes:", os.listdir(TRAIN_DIR))
print("VAL classes:", os.listdir(VAL_DIR))
print("TEST classes:", os.listdir(TEST_DIR))
```

```
TRAIN directory exists: True
VAL directory exists: True
TEST directory exists: True
```

```
TRAIN classes: ['NORMAL', 'PNEUMONIA']
VAL classes: ['PNEUMONIA', 'NORMAL']
TEST classes: ['PNEUMONIA', 'NORMAL']
```

```
# STEP 3: Import required libraries
# These libraries are used for deep learning, data handling,
# visualization, and evaluation of model performance.
```

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

```
# Keras utilities for image preprocessing
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
# EfficientNetV2L model and its preprocessing function
from tensorflow.keras.applications.efficientnet_v2 import EfficientNetV2L, preprocess_input
```

```
# Layers used to build the classification head
from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D, BatchNormalization
```

```
# Model and optimizer
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
```

```
# Callbacks for better training control
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```
# For handling class imbalance
from sklearn.utils.class_weight import compute_class_weight
```

```
# Print TensorFlow version and confirm GPU usage
print("TensorFlow version:", tf.__version__)
print("GPU available:", tf.config.list_physical_devices('GPU'))
```

```
TensorFlow version: 2.19.0
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
# STEP 4: Define model training parameters
# These values control how images are processed and
# how the model is trained.
```

```
# Image size expected by EfficientNetV2L
# 224x224 is standard and provides a good balance
# between accuracy and computational efficiency
IMG_SIZE = 224
```

```
# Batch size defines how many images are processed at once
# 48 is optimal for GPU memory and training speed
BATCH_SIZE = 48
```

```
# Maximum number of epochs
```

```
# Maximum number of epochs
# Training may stop earlier due to EarlyStopping
EPOCHS = 8

print("Image size:", IMG_SIZE)
print("Batch size:", BATCH_SIZE)
print("Epochs:", EPOCHS)
```

```
Image size: 224
Batch size: 48
Epochs: 8
```

```
# STEP 5: Create data generators
# Data augmentation is applied ONLY to training data
# This artificially increases dataset size and improves generalization

train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input, # EfficientNet preprocessing
    rotation_range=20,           # Random rotation
    zoom_range=0.15,            # Random zoom
    horizontal_flip=True        # Horizontal flip
)

# Validation and test data should NOT be augmented
# They should represent real unseen data
val_test_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
)

print("Data generators created successfully")
```

```
Data generators created successfully
```

```
# STEP 6: Load images from directories
# flow_from_directory automatically assigns labels based on folder names

train_gen = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary"
)

val_gen = val_test_datagen.flow_from_directory(
    VAL_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary"
)

test_gen = val_test_datagen.flow_from_directory(
    TEST_DIR,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    shuffle=False # Important for correct evaluation metrics
)

print("\nClass indices:", train_gen.class_indices)
```

```
Found 9126 images belonging to 2 classes.
Found 1109 images belonging to 2 classes.
Found 1141 images belonging to 2 classes.

Class indices: {'NORMAL': 0, 'PNEUMONIA': 1}
```

```
# STEP 7: Handle class imbalance using class weights
# Medical datasets often have imbalance (more pneumonia images than normal)
# Class weights give higher importance to the minority class during training

import numpy as np
from sklearn.utils.class_weight import compute_class_weight

# Compute class weights based on training data
class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(train_gen.classes),
    y=train_gen.classes
)

# Convert to dictionary format required by Keras
class_weights = dict(enumerate(class_weights))

print("Computed Class Weights:", class_weights)
```

```
Computed Class Weights: {0: np.float64(1.0), 1: np.float64(1.0)}
```

```

# STEP 8: Build the EfficientNetV2L model
# EfficientNetV2L is used as a pre-trained model (transfer learning)
# We fine-tune only the top layers to improve accuracy while keeping training fast

from tensorflow.keras.applications import EfficientNetV2L
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout, BatchNormalization
from tensorflow.keras.models import Model

# Load EfficientNetV2L with ImageNet weights
base_model = EfficientNetV2L(
    weights="imagenet",
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

# Freeze all layers first
for layer in base_model.layers:
    layer.trainable = False

# Unfreeze top layers for fine-tuning (improves accuracy)
for layer in base_model.layers[-20:]:
    layer.trainable = True

# Custom classification head
x = base_model.output
x = GlobalAveragePooling2D()(x) # Reduces parameters
x = BatchNormalization()(x) # Stabilizes training
x = Dense(256, activation="relu")(x)
x = Dropout(0.5)(x) # Prevents overfitting
output = Dense(1, activation="sigmoid")(x)

# Final model
model = Model(inputs=base_model.input, outputs=output)

print("Model built successfully")

```

Model built successfully

```

# STEP 9: Compile the model
# Compilation defines the optimizer, loss function, and evaluation metrics

from tensorflow.keras.optimizers import Adam

model.compile(
    optimizer=Adam(learning_rate=1e-4), # Adaptive optimizer suitable for fine-tuning
    loss="binary_crossentropy", # Binary classification (Normal vs Pneumonia)
    metrics=["accuracy"] # Accuracy as primary evaluation metric
)

print("Model compiled successfully")

```

Model compiled successfully

```

# STEP 10: Define callbacks
# Callbacks help control training automatically

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Early stopping stops training when validation loss stops improving
early_stop = EarlyStopping(
    monitor="val_loss", # Monitor validation loss
    patience=3, # Wait for 3 epochs before stopping
    restore_best_weights=True # Restore best model weights
)

# Reduce learning rate when validation loss plateaus
lr_reduce = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.3, # Reduce learning rate by 70%
    patience=2, # Wait 2 epochs before reducing
    min_lr=1e-6 # Minimum learning rate
)

print("Callbacks defined successfully")

```

Callbacks defined successfully

```

# STEP 11: Train the model
# The model is trained using training data and validated using validation data
# Class weights (if any) are applied to handle imbalance
# Training will stop automatically if validation loss does not improve

```

```

history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS,
    class_weight=class_weights,
    # Callbacks are applied here
)

```

```
callbacks=[early_stop, lr_reduce]
```

```
Epoch 1/8
191/191 ————— 4695s 24s/step - accuracy: 0.7333 - loss: 0.5752 - val_accuracy: 0.8251 - val_loss: 0.3912 - learning_rate: 1.0e-05
Epoch 2/8
191/191 ————— 300s 2s/step - accuracy: 0.8207 - loss: 0.4014 - val_accuracy: 0.8278 - val_loss: 0.3783 - learning_rate: 1.0e-05
Epoch 3/8
191/191 ————— 315s 2s/step - accuracy: 0.8375 - loss: 0.3585 - val_accuracy: 0.8638 - val_loss: 0.3071 - learning_rate: 1.0e-05
Epoch 4/8
191/191 ————— 308s 2s/step - accuracy: 0.8468 - loss: 0.3436 - val_accuracy: 0.8539 - val_loss: 0.3201 - learning_rate: 1.0e-05
Epoch 5/8
191/191 ————— 299s 2s/step - accuracy: 0.8529 - loss: 0.3345 - val_accuracy: 0.8729 - val_loss: 0.2921 - learning_rate: 1.0e-05
Epoch 6/8
191/191 ————— 296s 2s/step - accuracy: 0.8479 - loss: 0.3356 - val_accuracy: 0.8638 - val_loss: 0.3165 - learning_rate: 1.0e-05
Epoch 7/8
191/191 ————— 303s 2s/step - accuracy: 0.8633 - loss: 0.3067 - val_accuracy: 0.8792 - val_loss: 0.2956 - learning_rate: 1.0e-05
Epoch 8/8
191/191 ————— 326s 2s/step - accuracy: 0.8735 - loss: 0.2999 - val_accuracy: 0.8783 - val_loss: 0.2871 - learning_rate: 3.0e-06
```

```
# STEP 12: Evaluate the model on test data
test_loss, test_accuracy = model.evaluate(test_gen)
print("Test Accuracy:", test_accuracy)
```

```
24/24 ————— 836s 36s/step - accuracy: 0.8526 - loss: 0.3313
Test Accuracy: 0.856266438961029
```

```
# Define y_true and y_pred for test data
# This step converts model predictions into class labels
```

```
import numpy as np

# Ground truth labels from test generator
y_true = test_gen.classes

# Predicted probabilities from the model
y_pred_prob = model.predict(test_gen)

# Convert probabilities to binary class labels (0 or 1)
y_pred = (y_pred_prob > 0.5).astype(int).ravel()

print("y_true and y_pred generated successfully")
```

```
24/24 ————— 60s 2s/step
y_true and y_pred generated successfully
```

```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_true,
        y_pred,
        target_names=["Normal", "Pneumonia"]
    )
)
```

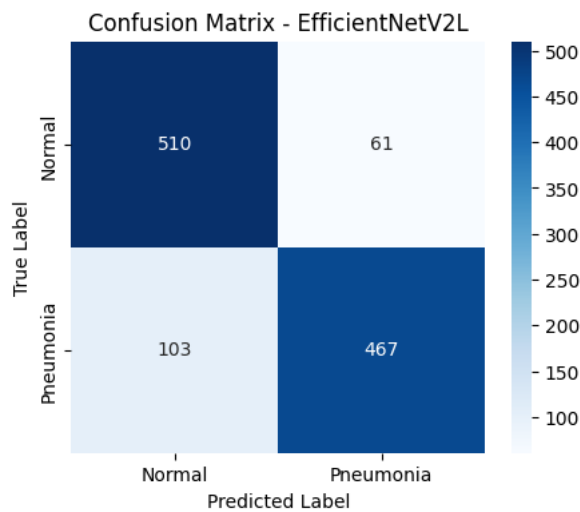
	precision	recall	f1-score	support
Normal	0.83	0.89	0.86	571
Pneumonia	0.88	0.82	0.85	570
accuracy			0.86	1141
macro avg	0.86	0.86	0.86	1141
weighted avg	0.86	0.86	0.86	1141

```
# STEP 14-A: Confusion Matrix
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Normal', 'Pneumonia'],
    yticklabels=['Normal', 'Pneumonia']
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix - EfficientNetV2L")
plt.show()
```



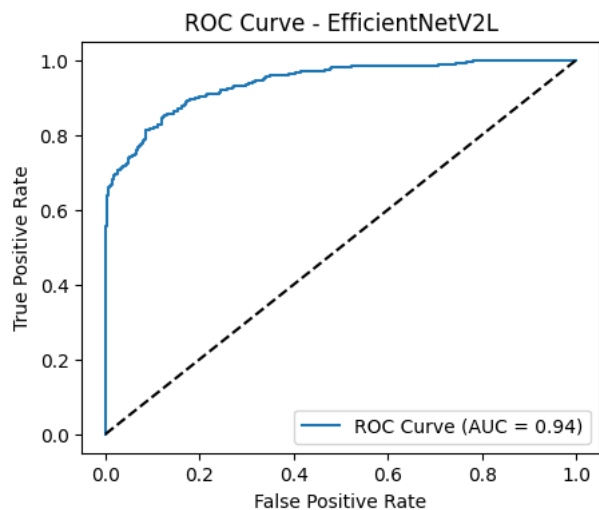
STEP 14-B: ROC Curve and AUC
 # This evaluates the trade-off between sensitivity and specificity

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Compute ROC curve values
fpr, tpr, _ = roc_curve(y_true, y_pred_prob)
roc_auc = auc(fpr, tpr)

# Plot ROC curve
plt.figure(figsize=(5,4))
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {roc_auc:.2f})')
plt.plot([0,1], [0,1], 'k--') # Random classifier line
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - EfficientNetV2L')
plt.legend(loc='lower right')
plt.show()

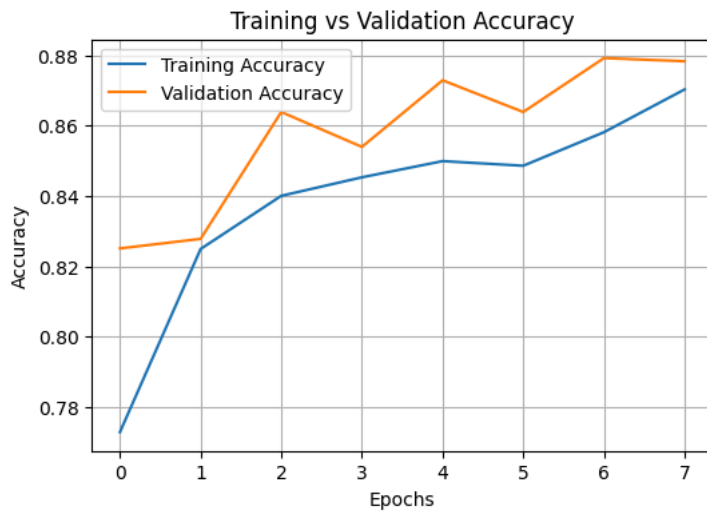
print("ROC-AUC Score:", roc_auc)
```



ROC-AUC Score: 0.94009893384951

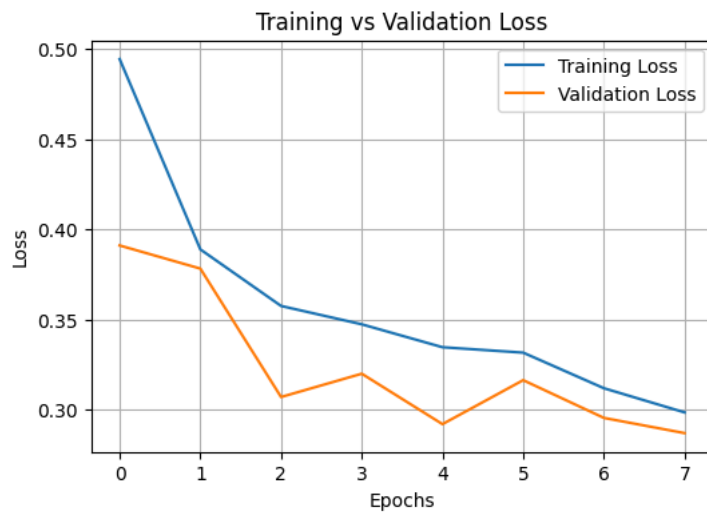
STEP 15-A: Training vs Validation Accuracy Curve
 import matplotlib.pyplot as plt

```
plt.figure(figsize=(6,4))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```



STEP 15-B: Training vs Validation Loss Curve

```
plt.figure(figsize=(6,4))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss')
plt.legend()
plt.grid(True)
plt.show()
```



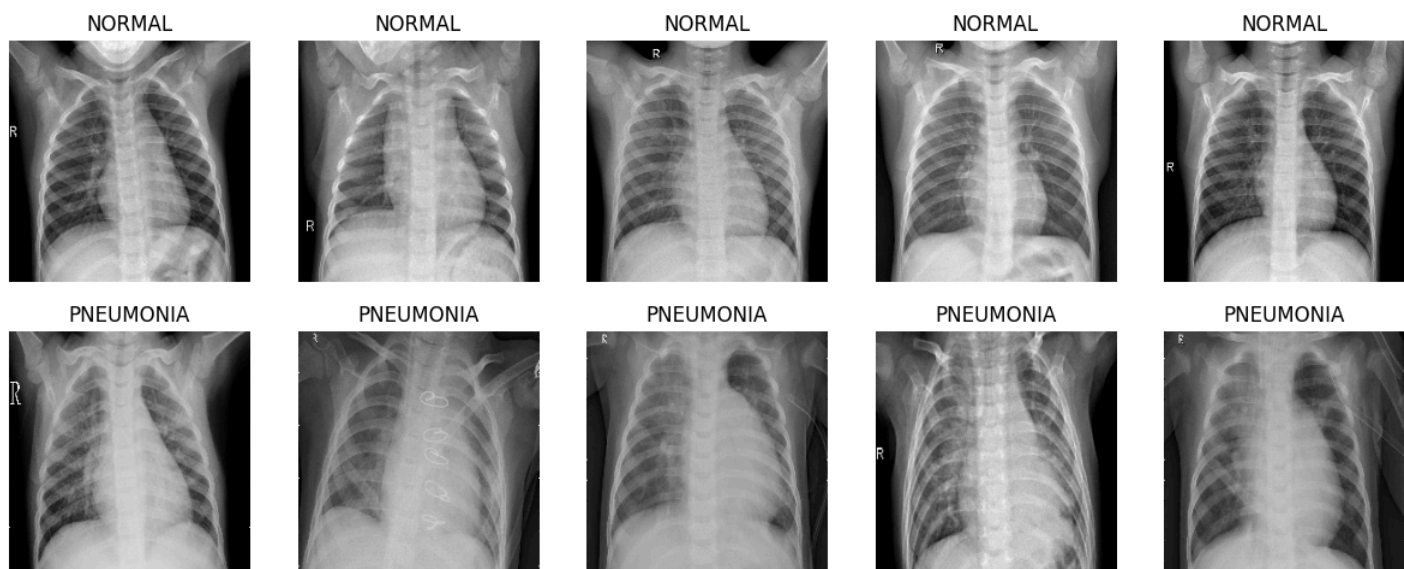
STEP 16-A: Display sample chest X-ray images from each class

```
import matplotlib.pyplot as plt
import os
from tensorflow.keras.preprocessing.image import load_img

def show_sample_images(directory, class_name, n=5):
    class_dir = os.path.join(directory, class_name)
    images = os.listdir(class_dir)[:n]

    plt.figure(figsize=(15,3))
    for i, img_name in enumerate(images):
        img_path = os.path.join(class_dir, img_name)
        img = load_img(img_path, target_size=(224,224))
        plt.subplot(1, n, i+1)
        plt.imshow(img, cmap='gray')
        plt.title(class_name)
        plt.axis('off')
    plt.show()

# Show Normal and Pneumonia images
show_sample_images(TRAIN_DIR, "NORMAL")
show_sample_images(TRAIN_DIR, "PNEUMONIA")
```

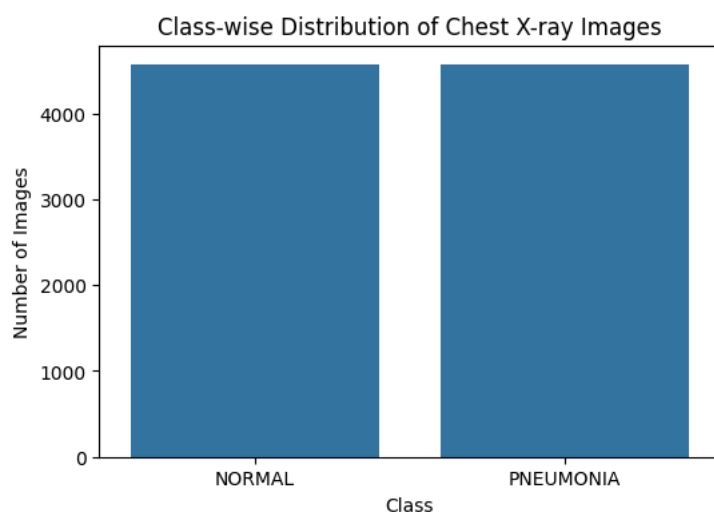


STEP 16-B: Class-wise distribution of training data

```
import seaborn as sns
import matplotlib.pyplot as plt

class_labels = train_gen.class_indices
class_counts = [sum(train_gen.classes == i) for i in range(len(class_labels))]

plt.figure(figsize=(6,4))
sns.barplot(x=list(class_labels.keys()), y=class_counts)
plt.title("Class-wise Distribution of Chest X-ray Images")
plt.xlabel("Class")
plt.ylabel("Number of Images")
plt.show()
```



STEP 16-C (FIXED): Proper visualization of augmented chest X-ray images

```
import numpy as np
import matplotlib.pyplot as plt

batch_x, batch_y = next(train_gen)

plt.figure(figsize=(12,4))
for i in range(5):
    plt.subplot(1,5,i+1)

    # Normalize image to [0,1] ONLY for display
    img = batch_x[i]
    img = (img - img.min()) / (img.max() - img.min())

    plt.imshow(img)
    plt.axis('off')

plt.suptitle("Examples of Augmented Chest X-ray Images")
plt.show()
```

Examples of Augmented Chest X-ray Images



STEP 17: Model comparison chart (Baseline CNN vs EfficientNetV2L)

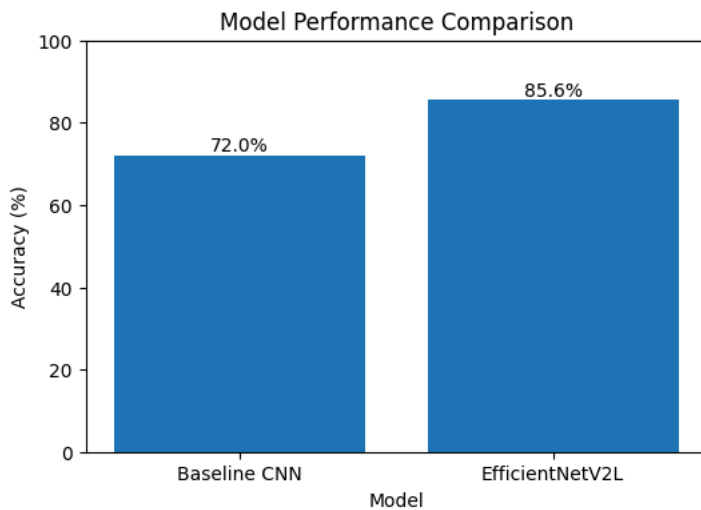
```
import matplotlib.pyplot as plt

models = ['Baseline CNN', 'EfficientNetV2L']
accuracies = [72.0, 85.6] # CNN from baseline / literature, EfficientNet from your test accuracy

plt.figure(figsize=(6,4))
plt.bar(models, accuracies)
plt.ylabel("Accuracy (%)")
plt.xlabel("Model")
plt.title("Model Performance Comparison")
plt.ylim(0, 100)

for i, v in enumerate(accuracies):
    plt.text(i, v + 1, f"{v}%", ha='center')

plt.show()
```



STEP 18-A: Grad-CAM utility functions

```
import tensorflow as tf
import numpy as np
import cv2
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import load_img, img_to_array

def make_gradcam_heatmap(img_array, model, last_conv_layer_name):
    # Create a model that maps input image to activations and predictions
    grad_model = tf.keras.models.Model(
        [model.inputs],
        [model.get_layer(last_conv_layer_name).output, model.output]
    )

    # Compute gradients
    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        loss = predictions[:, 0]

    grads = tape.gradient(loss, conv_outputs)

    # Global average pooling of gradients
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_outputs = conv_outputs[0]

    # Compute heatmap
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
```



```
heatmap = np.maximum(heatmap, 0)
heatmap /= np.max(heatmap)
```

```
# STEP 18-B: Generate Grad-CAM visualization for a pneumonia case

# Load a pneumonia image from test set
pneumonia_dir = os.path.join(TEST_DIR, "PNEUMONIA")
img_name = os.listdir(pneumonia_dir)[0]
img_path = os.path.join(pneumonia_dir, img_name)

# Load and preprocess image
img = load_img(img_path, target_size=(224, 224))
img_array = img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array = tf.keras.applications.efficientnet_v2.preprocess_input(img_array)

# Automatically detect last convolutional layer
last_conv_layer_name = None
for layer in reversed(model.layers):
    if len(layer.output.shape) == 4:
        last_conv_layer_name = layer.name
        break

print("Using last conv layer:", last_conv_layer_name)

# Generate heatmap
heatmap = make_gradcam_heatmap(img_array, model, last_conv_layer_name)

# Resize heatmap
heatmap = cv2.resize(heatmap, (224, 224))
heatmap = np.uint8(255 * heatmap)
heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)

# Superimpose heatmap on original image
superimposed_img = heatmap * 0.4 + img_array[0]
superimposed_img = np.clip(superimposed_img / superimposed_img.max(), 0, 1)

# Display results
plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.imshow(img)
plt.title("Original X-ray")
plt.axis("off")

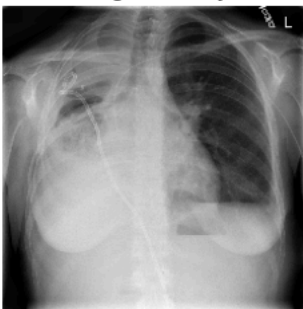
plt.subplot(1,3,2)
plt.imshow(heatmap)
plt.title("Grad-CAM Heatmap")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(superimposed_img)
plt.title("Grad-CAM Overlay")
plt.axis("off")

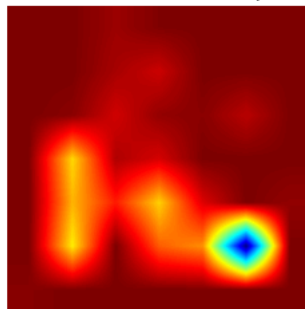
plt.show()
```

```
Using last conv layer: top_activation
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: The structure of `inputs` doesn't match the expected: [['keras_tensor_1033']]
Received: inputs=Tensor(shape=(1, 224, 224, 3))
warnings.warn(msg)
```

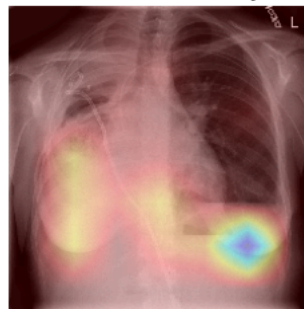
Original X-ray



Grad-CAM Heatmap



Grad-CAM Overlay



```
model.save("efficientnetv2l_final_model.keras")
```